QUICK START TO JAVA

– Building on top of introductory JavaScript programming experience

Contents

1 Moving from JavaScript to Java 2
2 Comparing a Java program to a JavaScript program 2
3 Creating, compiling, and running a Java program in Eclipse 5
4 Code layout and automatic source code formatting in Eclipse 5
5 Java naming convention 6
6 Symbols in a Java source text, whitespace, comments 6
7 Inputting data from the user 7
8 Printing to the console, formatting decimal values for printing 8
9 Handling invalid numeric input (exception handling) 9
10 Basic data types and variables 10
11 Scopes of variables 11
12 Basic operators, expressions, type conversions, and surprises in calculation results 12
13 Conditional statements in Java 15
14 Writing loops in Java 16
15 Methods *in a stand-alone Java program class* 17
16 Array handling 21
17 String handling and regular expressions 22
18 All primitive data types and internal representations 26
19 Packages 28
List of Java keywords and Java API classes used in the examples 29

More Online Java Learning

Oracle's Java tutorials	http://docs.oracle.com/javase/tutorial
–> Learning the Java Language	http://docs.oracle.com/javase/tutorial/java
Java language specification:	https://docs.oracle.com/javase/specs
Java API documentation	https://docs.oracle.com/javase/8/docs/api https://docs.oracle.com/en/java/javase/24/docs/api

The following site has excellent online Java learning material: http://mooc.fi/courses/2013/programming-part-1/material.html

1 Moving from JavaScript to Java

To say it in short: *Java is to JavaScript as cat is to catfish*.

Java and JavaScript have very different origins.

- **Java** was designed in 1991 for intelligent consumer-electronic devices (interactive television etc.) by a team led by James Gosling at Sun Microsystems.
- The first version of **JavaScript** (originally named *Mocha*, then *LiveScript*) was created in 1995 by Brendan Eich at Netscape "to control almost everything in the Netscape [web] browser".

The very first steps in moving from JavaScript to Java are quite straightforward. Both languages have borrowed several details from the same source. Examples of practically identical details are

- The basic ***control statements*** (conditional statements, loop statements etc.)
- Most of the ***basic operators***
- The basic ***punctuators*** (the use of several types brackets, semicolons etc.)

Otherwise, these two programming languages are very different. **Java** is an object-oriented general-purpose programming language. Java programs consist of pieces called *classes*. You will typically use Java classes that you create yourself, classes from existing class libraries, classes that others make available for you, and application frameworks. See also: Java in Wikipedia.

Here is the first Java program example. The first code lines are very different from before, but the code inside the main method should look quite familiar.

Java source code of a simple *stand-alone* Java program

PowersOfTwoProgram.java

```
public class PowersOfTwoProgram {

    public static void main(String[] args) {
        int power = 1;

        while (power < 50) {
            System.out.print(power + " ");
            power = power * 2;
        }
    }
}
```

Question: Can you figure out what the program above prints to the console?

2 Comparing a Java program to a JavaScript program

Suppose that the program should print the following five lines of text *to the console*:

```
1 Hello!
2 Hello!
3 Hello!
4 Hello!
5 Hello!
```

Let's compare the JavaScript source code to the Java source code to see the similarities and differences in these source codes.

JavaScript source code

HelloProgram.html	**HelloProgram.js**

`<!DOCTYPE html>`
`<html lang="en">`
`<head>`
`  <meta charset="UTF-8" />`
`  <title>Hello Program</title>`
`</head>`
`<body>`
`  <script src="HelloProgram.js">`
`  </script>`
`</body>`
`</html>`
	`function main() {`
	
	`  var greetingText = "Hello!";`
	
	`  for (var i = 1; i <= 5; i++) {`
	`  console.log(i + " " + greetingText);`
	`  }`
	`}`
	
	`main();`

Java source code of a simple *stand-alone* Java program

HelloProgram.java

```
public class HelloProgram {

    public static void main(String[] args) {

        String greetingText = "Hello!";

        for (int i = 1; i <= 5; i++) {
            System.out.println(i + " " + greetingText);
        }
    }
}
```

Similarities in the codes

- The use of punctuators (brackets, semicolons etc.)
- The loop statement syntax
- The operators (assignment, comparison, incrementation, string concatenation)
- The way how literals are written (text enclosed in double quotes, plain numbers)

What is different in the Java source code?

- The file name extension must be ***.java***
- All ***Java code is written inside class declarations***[1]. In the example, the program class is named *HelloProgram*. The program class must be a public class. The file name must be the same as the program class name.
- A stand-alone ***Java program must contain a main method*** whose header looks exactly like the one in the example[2]. The ***program execution starts from the main method***. The term *method* is used in here instead of the term *function*.
- Java is a ***strongly-typed language***. Therefore, each variable is defined with a fixed data type (String, int).
- `System.out.println` is used to print to the console.

[1] Often, a stand-alone Java program consists of the program class (the class that has the main method) and several other classes (that do not have the main method). NB! We discuss object-oriented programming and classes for objects later.

[2] The first line (*signature*) of the **main** method must be declared **public**, **static**, and **void**. The method must accept a single parameter that is a *reference to an array of Strings*. The ***public*** access modifier allows the Java run-time engine to access the main method. The ***static*** keyword allows the Java run-time engine access the method without having to instantiate (*to create on object from*) the class. The **void** keyword specifies that the method does not return a value. **String[] args** defines the *args* reference variable of type 'array of Strings' .

What is different in the running environment / running platform?

JavaScript running environment / running platform

- The **JavaScript** program is executed ***in the web browser***.
- The JavaScript source code is saved as text file with the ***.js*** extension.
- When the program is to be executed, the web browser loads and ***interprets*** the ***JavaScript source code***.
- The JavaScript program is distributed in its human-readable source ***code.***
- Syntax errors in the source code make the program crash when it is executed.

Java running environment / running platform

- The **Java** program is executed ***in the Java Virtual Machine***. Java Runtime Environment (**JRE**) is a software package that is needed to be installed before you can run a Java program. It includes the *Java Virtual Machine* (**JVM**) together with the *Java Class Library* (**Java API**).

- Before a Java program can be executed, the human-readable Java source code must be ***compiled*** into machine-readable instructions (*bytecode*).
- When the program is to be executed, the Java Virtual Machine loads and interprets the machine-readable bytecode.
- The Java program is distributed in its machine-readable ***bytecode*** (***.class*** files).
- Syntax errors in the source code never make the program crash. If the Java compiler finds one or more syntax errors when the source code is being compiled, then it fails the compilation and shows one or more error messages.

Compiling and running a simple stand-alone Java program from the command line

1. The Java source code is saved as a text file with the ***.java*** extension (*HelloProgram.java*)

```
 Directory of D:\JavaExamples

08.06.2017  09:59    <DIR>          .
08.06.2017  09:59    <DIR>          ..
08.06.2017  09:44               230 HelloProgram.java
               1 File(s)            230 bytes
```

2. The Java source code (***.java*** file) must be compiled into portable bytecode (***.class*** file). ***Compiling*** a Java program means that the Java complier (***javac***) takes the human-readable source code and converts it to machine-readable *bytecode* instructions for the Java Virtual Machine.

```
D:\JavaExamples>javac HelloProgram.java
```

As the result of a successful compilation of *HelloProgram**.java*** the compiler saves the bytecode in a file named *HelloProgram**.class***

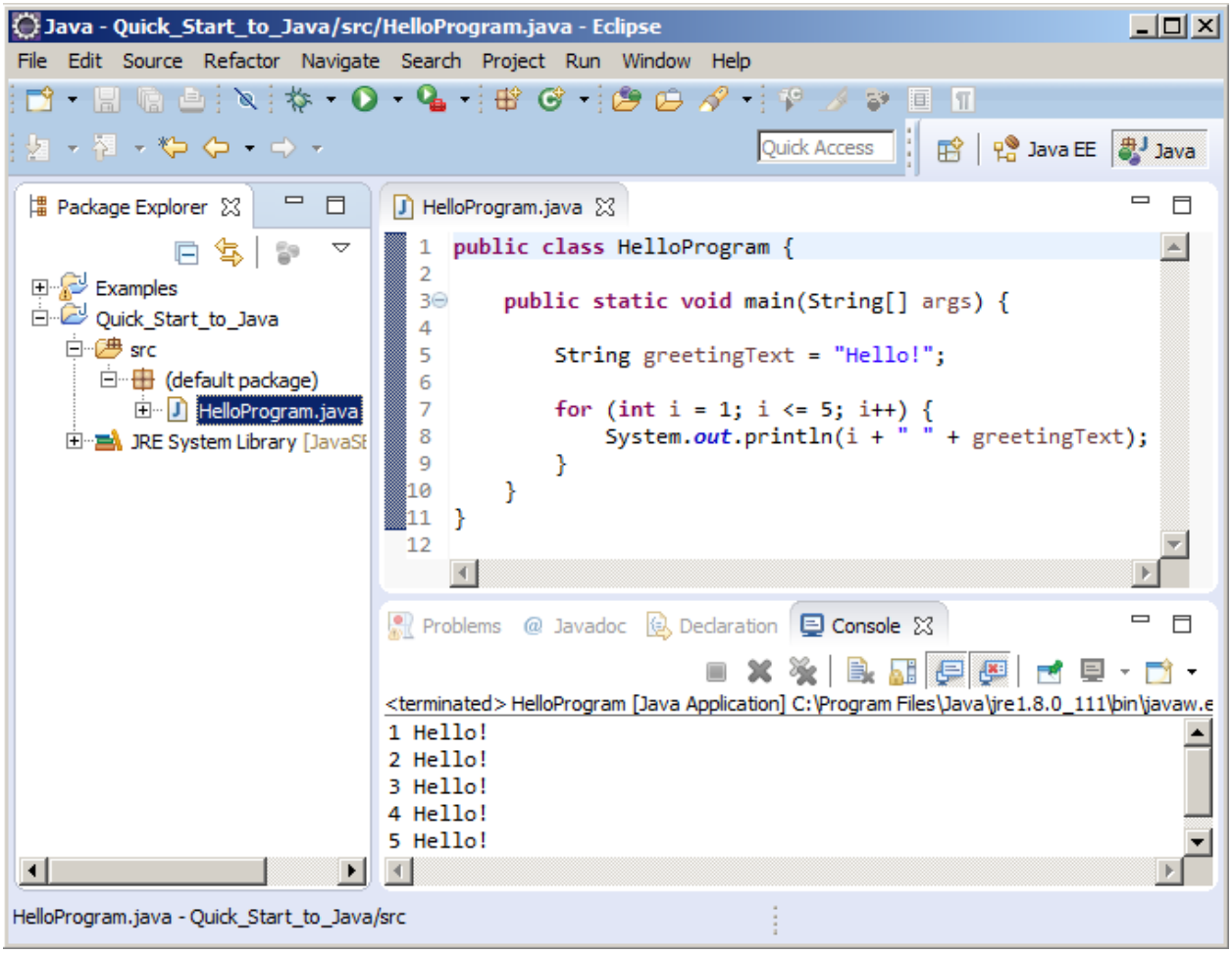
```
 Directory of D:\JavaExamples

08.06.2017  10:03    <DIR>          .
08.06.2017  10:03    <DIR>          ..
08.06.2017  10:03               722 HelloProgram.class
08.06.2017  09:44               230 HelloProgram.java
               2 File(s)            952 bytes
```

3. Finally, the compiled version of the program can be executed by invoking the Java Virtual Machine with the name of the program file (***.class*** file).

```
D:\JavaExamples>java HelloProgram
1 Hello!
2 Hello!
3 Hello!
4 Hello!
5 Hello!
```

3 Creating, compiling, and running a Java program in Eclipse



In the screenshot above, you see the Java source code in the *code editor* tab and the console output in the *Console* tab.

See the **Eclipse Quick Start** guide[3] on how to create a new ***Java project***, create a new ***Java program class*** in the project, compile, and run the Java program. In the above example, the name of the Java project is *Quick_Start_to_Java* and the name of the program class is *HelloProgram*.

Please make sure that you learn how to use the *context-sensitive* ***auto-complete feature*** (*Content assist*) in Eclipse. This tool helps you to ***write code faster and more efficiently***. It activates when you press **Ctrl** and **Spacebar** simultaneously.

4 Code layout and automatic source code formatting in Eclipse

See the code examples for the code layout to be followed on this course. In Eclipse, you can easily format your code as below to make sure that the ***indentation***, ***use of whitespace***, and alignment of ***braces*** "{" and "}" are ok.

In Eclipse toolbar, select **Source** > **Format** OR press **Ctrl** + **Shift** + **F**

The use of systematic indentation is crucial for the readability of program code. You are always expected to write code that is easy to read and understand. Fulfilling these criteria makes programs easy to maintain, reduces potential errors and saves time and money later.

Please study carefully ***the use of empty lines*** in the code examples.

[3] The 'Eclipse Quick Start' guide is available on the course pages.

5 Java naming convention

In programming, user-defined names are called ***identifiers***. In Java, you normally use letters a...z and A...Z, digits (0...9) and underscore characters (_) in identifiers. Java is case-sensitive.

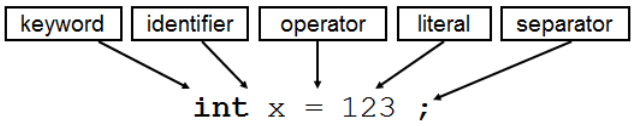
A user-defined name should begin with a letter. The rest of the name can consist of letters, numbers and underscore characters.

- *Class names* begin with a capital letter
- *Variable names* and *method names* begin with a lowercase letter

Use *camel case* (medial capitals). In camel case the first letter of a name is either lowercase or uppercase and the first letter of each subsequent concatenated word is capitalized. Other letters in the name are lowercase.

6 Symbols in a Java source text, whitespace, comments

The Java Language specification classifies the symbols in a Java source code as follows: ***identifiers***, ***keywords***, ***operators***, ***literals***, *and* ***separators***.



Identifiers are *user-definable names* of variables, methods, classes etc.

Keywords (e.g. `int`, `double`, `if`, `while`, `for`, `return`) are words that are reserved for special purposes in the programming language specification[4]. Keywords cannot be used as *identifiers*. There are 51 keywords in Java.

Operators indicate which operations to apply to their *operands.* There are 38 operators in Java.

Literals are source code representations of fixed values.

Separators (punctuators) separate some language features from other language features in source code. For example, semicolon is used to separate one statement from another. In Java, there are the following separators: `( ) { } [ ] ; , .`

White space (*whitespace*) is any non-visible character or series of non-visible characters that represent horizontal or vertical space in typography. The most common whitespace characters may be typed via the *space bar,* the *tab key*, and the *enter key*. Each of these characters has its own character code that is saved in the source text file.

A ***comment*** is a programmer-readable explanation in the source code. Generally, comments are ignored by compilers and interpreters. Java supports the following three kinds of comments:

1. *Single-line comment* The compiler ignores everything from **//** to the end of the line.
2. *Multi-line comment* The compiler ignores everything from **/*** to ***/**
3. *Javadoc comment* The compiler ignores everything from **/**** to ***/**

The ***Javadoc*** tool[5] reads javadoc comments when we use it to prepare automatically generated documentation.

[4] See the Java language specification for a complete list of keywords and operators. NB! Class names are identifiers, not keywords. For example, it is technically possible to replace the default *System* class with your own *System* class.

[5] See https://en.wikipedia.org/wiki/Javadoc

7 Inputting data from the user

A very simple ***stand-alone Java program*** reads user input from console. That is, the user writes a line of text and submits the whole line by pressing the *Enter* key.

Unfortunately, Java does not provide any sophisticated way to read user input from console. Typically, the simple *text scanning tool* (the `Scanner` class) is used for this purpose.

1. The `Scanner` class is included in the standard Java class library. It must be imported from the `java.util` package. Otherwise, the Java compiler does not find it.
2. Before you can start reading user input you must create an object from the `Scanner` class.
3. The `nextLine()` method of a `Scanner` object takes input from the user at the command line and returns the whole line as a single `String` when the user presses the *Enter* key.
4. **NB!** If you want to input a numeric value, then you should first input a `String` and then convert it to a numeric value[6]. Please see also the footnote.
5. Finally, close the scanner to get rid of the warning "*Resource leak: 'input' is never closed*".

AgeProgram.java

```
import java.util.Scanner;                                   // 1.

public class AgeProgram {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);             // 2.

        System.out.print("Enter your first name: ");
        String firstName = input.nextLine();                // 3.

        System.out.print("Enter your age: ");
        int age = Integer.parseInt(input.nextLine());       // 4.

        if (age < 18) {
            System.out.println(firstName + ", you are a minor.");
        } else {
            System.out.println(firstName + ", you are an adult.");
        }

        input.close();                                      // 5.
    }
}
```

In Eclipse, the program runs as follows:

```
Console   Problems   @ Javadoc   Declaration
<terminated> AgeProgram [Java Application] C:\Program Files\Ja
Enter your first name: James
Enter your age: 19
James, you are an adult.
```

[6] **IMPORTANT**: This is a *robust way* to handle the user input.

The Scanner class also provides methods nextInt() and nextDouble(). Please notice that these methods do not directly input any numeric data. Instead, they scan the inputted text in the input buffer and try to convert the next available token in the buffer to an int or double. Please do NOT use these methods!

The problem is that ***nextInt() and nextDouble() do not consume the newline character of the input***. That is, if you call the nextLine() method right after you have called either nextInt() or nextDouble(), then the nextLine() method just consumes the newline character of the previous input and ***it totally skips the new input***.

To input a ***decimal value*** is quite tricky. To make it flexible we can do it as below to let the user to enter a comma or a dot as the decimal separator.

```
double price = Double.parseDouble(input.nextLine().replace(',', '.'));
```

I agree, this is not neat at all, but this way ***you avoid some time-consuming problems*** (*especially in the Viope environment*). Writing a small method as below improves the readability of the code.

PriceWithVAT.java

```
import java.util.Scanner;

public class PriceWithVAT {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter price: ");
        double price = readDouble(input);

        System.out.println("Price with VAT is: " + 1.24 * price);
    }

    private static double readDouble(Scanner input) {
        return Double.parseDouble(input.nextLine().replace(',', '.'));
    }
}
```

8 Printing to the console, formatting decimal values for printing

You can use `System.out.`**`println`** (*moves the cursor to a new line*)[7] and `System.out.`**`print`** (*does not move the cursor to a new line*) to print to the console. To ***set the exact number of decimal places*** to be printed you can use the **`DecimalFormat`** class as shown below. The `format` method of the `DecimalFormat` class formats a decimal value into a *locale-specific* String[8].

DecimalFormattingExample.java

```
import java.text.DecimalFormat;

public class DecimalFormattingExample {

    public static void main(String[] args) {
        DecimalFormat oneDecimal = new DecimalFormat("0.0");
        DecimalFormat twoDecimals = new DecimalFormat("0.00");

        double x = 4.67212;
        double y = 3.1415926;

        System.out.println("x is " + oneDecimal.format(x));
        System.out.println("y is " + twoDecimals.format(y));
    }
}
```

The program prints the below to the console. **NB!** Depending on the ***localization settings*** of the computer's operation system, either ***dot*** or ***comma*** is shown as the decimal separator.

```
x is 4.7
y is 3.14
```

```
x is 4,7
y is 3,14
```

[7] (Advanced): `System` is a Java class and *`out`* is a public static variable in the System class. The *`out`* variable holds a reference to a `PrintStream` object. `println` is a method of the `PrintStream` object.

[8] *Locale-specific* means here that the formatting operation retrieves the current localization settings from the host environment. Depending on the localization settings, either comma or dot is used as the decimal separator.

9 Handling invalid numeric input (exception handling)

Suppose that the program inputs an integer as below.

```
int x = Integer.parseInt(input.nextLine());
```

If the conversion from the input string to a numeric value fails, then the Java run-time engine aborts the program and prints an error message. The message explains *what kind of an error it was* and *where it occurred*[9]. In Java, *run-time errors* are called ***exceptions***.

```
Enter an integer: FIVE
```

Exception in thread "main" java.lang.**NumberFormatException**: **For input string: "FIVE"**
at java.lang.NumberFormatException.forInputString(Unknown Source)
at java.lang.**Integer.parseInt**(Unknown Source)
at UserInputExample.main(**UserInputExample.java:8**)

Java provides the ***exception handling*** mechanism (`try/catch`) for handling run-time errors.

- The **`try`** clause establishes a block of code that is to have its exceptions handled.
- A **`catch`** clause specifies *what type of exceptions it will handle* and *the code to be executed if such an exception occurs* when the code in the **`try`** block is being executed.

If an operation in the **try** block fails, then (1) the execution of the code in the **try** block ends and (2) the program execution jumps to the **catch** block. (3) After the code in the **catch** block is executed the program execution moves out from the try/catch structure and continues normally.

In the example below, the conversion from a String to an int fails if the user enters an invalid integer. If the conversion fails, the method `parseInt` signals this by ***throwing*** a `NumberFormatException`. To prevent the program to crash we use try/catch and specify that the `catch` clause should *catch* and handle exceptions of the type `NumberFormatException`[10] .

```
try {
    System.out.print("Enter an integer: ");
    int x = Integer.parseInt(input.nextLine());

    System.out.println(x + " is a valid integer.");

} catch (NumberFormatException nfe) {
    System.out.println("Not a valid integer!");
}

System.out.println("Bye!");
```

If the user enters a valid integer, then the program runs as follows:

```
Enter an integer: 12
12 is a valid integer.
Bye!
```

If the user enters an invalid integer, then the program runs as follows:

```
Enter an integer: FIVE
Not a valid integer!
Bye!
```

If you want to **display the *inputted text* in an error message later**, then you must save the text to a separate variable. Then, you convert the text to an int and save the result to another variable.

```
String text = input.nextLine();
int x = Integer.parseInt(text);
```

[9] The source code line number is shown at the end of the message.

[10] There are many different exception types available for different types of errors. In this introductory material, we just cover how to handle a number format exception in a type conversion.

In the example (see the catch clause), ***nfe*** is a reference variable. For example, you can print the error message *generated by the failed method* as follows: `System.out.println(nfe.getMessage());`

10 Basic data types and variables

Java is a ***strongly typed*** programming language. That is, each variable must be defined with a fixed data type. The type of a variable determines the range of values the variable can have and the set of operations that are available for the type.

Below is a list of selected data types in Java. See chapter 18 for all primitive data types.

Data type category		Type in Java	Internal representation of the data type
Numeric data	*Integer*	`int`	32 bit *integer* Range: ± 2^{31} (-2147483648 ... +2147483647)
	Decimal number	`double`	64 bit double-precision IEEE 754 *floating point* 15 digits of precision, not accurate
Textual data	*String*	`String`	A string of **0 – n** Unicode characters (*in an object*)
	Single character	`char`	16 bit Unicode character (*exactly* ***one character***)
Truth value	*Boolean*	`boolean`	Allowed values: `true`, `false`

To declare a variable in Java, we write the data type name before the variable name.

```
int age = 21;                      // Integer only
int population = 11_687_123;       // '_' can be used in numbers to improve readability.
double percentage = 12.5;          // Decimal number
boolean isFound = false;           // Boolean value, either true or false
String firstName = "Frank";        // String, double quotes!
char letter = 'A';                 // Exactly one character, single quotes!
final double PI = 3.141592653589;  // A final variable may only be assigned to once. The
                                   // name of a final variable should be all uppercase.
```

Java does strict type checking and restricts operations that are not type compatible.

```
percentage = "24.75";              // Not allowed! "24.75" is a String, not a double
age = 10.5;                        // Not allowed! 10.5 is a double, not an integer
isFound = 0;                       // Not allowed! 0 is an int, not a boolean
firstName = 123;                   // Not allowed! 123 is an int, not a String
firstName = 'X';                   // Not allowed! 'X' is a char, not a String
letter = "A";                      // Not allowed! "A" is a String, not a char
```

A ***Java String*** contains an immutable[11] sequence of Unicode (UTF-16) characters. A String can contain any number of any characters[12].

String literals are always enclosed in *double quotes*. An empty string literal (zero characters) is written as follows: `""`. We can use character escape sequences in strings to include quotes (`\"`), newlines (`\n`), backslashes (`\\`) etc. Example: `System.out.println(" \"quotation\" ");`

In Java, we should NOT use operators to compare Strings[13]. Instead, we use methods as below.

```
if (email.isEmpty()) {
    System.out.println("Email is missing");
}

if (countryCode.equals("FI")) {
    System.out.println("Finland");
}
```

[11] That is, the content of the String cannot be modified once it is created. For example, `firstName.toUpperCase()` constructs and returns a new `String` instead of changing the existing content.
[12] Letters, digits, and special characters.
[13] (Advanced remarks): In Java, Strings are objects. But, Strings receive special treatment in Java, because they are used frequently in a program. String objects are automatically created from String literals, and we can concatenate Strings with the '+' operator. Note: `if (firstName == "John")` tests if the two object references are equal. The characters in the two Strings are not checked. Due to String literal pooling the operation `"abc" == "abc"` yields always `true`.

11 Scopes of variables

The ***scope** of a variable* is the part of the code where the variable can be referenced[14]. The *lifetime of a variable* is the time during which the variable stays in memory.

To say it in short,
a variable defined ***inside pair of braces*** "{" and "}" has *scope and lifetime* ***within the braces only.***
The scope *starts from the variable definition* and continues to the closing brace.

1. A variable defined ***outside of methods in a class declaration*** has scope within the class body.
2. A variable defined ***in a method declaration***[15] has scope within the method body only.
   The lifetime of the variable is the *execution of the method*.
3. A variable defined ***in a statement block***[16] has scope within the statement block only.
   The lifetime of the variable is the *execution of the statement block*.
4. A variable defined ***in the header of a for loop*** has scope within the loop only.
   The lifetime of the variable is the *execution of the for loop*.
5. A variable defined ***in a catch clause*** has scope within the catch clause only.
   The lifetime of the variable is the *execution of the catch clause*.

ScopeExample.java

```
public class ScopeExample {
    private static int aa = 1;                    // aa is visible in the whole class declaration

    public static void main(String[] args) {      // args is visible in the main method only
        int bb = aa + 2;                          // bb is visible in the main method only
                                                  // The scope of bb starts from its
        displayValues();                          // declaration and continues to the end
    }                                             // of the method.

    private static void displayValues() {
        int cc = 3;                               // cc is visible in this method only

        for (int i = 1; i <= 5; i++) {            // i is visible in this for loop only
            int dd = aa + cc + i;                 // dd is visible in the loop body only

            System.out.println(dd);
        }

        System.out.println(aa + cc);              // Here i and dd are not available any more
    }
}
```

In the example above , there are six variables. See the table below for scopes of these variables.

		The variable is visible in the ...		
Variable	*Scope level*	***main** method*	***displayValues** method*	***for loop***
aa	*class*	**YES**	**YES**	**YES**
args	*method*	**YES**	-	-
bb	*method*	**YES**	-	-
cc	*method*	-	**YES**	**YES**
dd	*block*	-	-	**YES**
i	*for loop*	-	-	**YES**

[14] That is, where the variable is "visible".
[15] In the method header or in the method body
[16] A ***statement block*** is a sequence of zero or more statements enclosed in braces. Statement blocks can be nested.

12 Basic operators, expressions, type conversions, and surprises in calculation results

An ***expression*** is something that can be evaluated to a value of a certain data type. In Java, the same expression evaluates always to a value of the same type. Expressions are constructed from ***operators*** and ***operands***. The operators of an expression indicate which operations to apply to the operands. Examples for operands are literals, variables, and method calls.

Usually, *almost any expression of type X* can be used where a single value of type X can be used. For example, if the syntax requires you to write an *int* in your code, you can usually write there almost *any expression that evaluates to an int*.

When an expression contains multiple operators, the ***precedence*** of the operators controls the *order in which the individual operators are evaluated*. For example, Java evaluates the expression `1 + 2 * 3` as `1 + (2 * 3)` because the multiplication operator has higher precedence than the addition operator. The evaluation order can be changed by using parentheses.

In the case of same level of the precedence, ***associativity*** *rules* define the order of evaluation (from *left to right* or *right to left*). In Java, most of the operators have *left associativity*. For example, in the expression `"1" + 2 + 3` string concatenation and addition are on the same level of the precedence. *Left associativity* means that the expression should be evaluated as `("1" + 2) + 3`, not `"1" + (2 + 3)`.

The table below shows some Java operators in decreasing order of precedence from top to bottom. In the table, there are nine precedence levels. Level 1 is the highest level of precedence.

#	Operators		Comments	Associativity

1	Postfix increment
	Postfix decrement
	Unary minus
	NOT
	Unary type cast
		`++`
		`--`
		`-`
		`!`
		`(` *type* `)`
			Increments the value by 1
			Decrements the value by 1
			`-(2 + 3) + 4` is `-1`
			`!true` is `false`
			`(int)3.99` is 3
				left to right
				left to right
				right to left
				right to left
				right to left

2	Multiplication
	Division
	Remainder
		`*`
		`/`
		`%`
			Note: The result of an ***integer division*** is an integer. For example, 1 / 2 is **0** (not 0.5).
			% yields the ***remainder*** of an integer division. 10 % 5 is 0 10 % 3 is 1 2 % 5 is 2
				left to right
				left to right
				left to right

3	Addition, Subtraction String concatenation	`+ -` `+`	The meaning of '+' depends on its operands	left to right left to right
4	Comparison	`> >= < <=`		left to right
5	Equality	`== !=`	Is equal to, Is not equal to	left to right
6	AND	`&&`	Note: *Short-circuit evaluation*[17]	left to right
7	OR	`||`	Note: *Short-circuit evaluation*	left to right
8	Conditional operator	`?:`	`maxValue = (a > b) ? a : b;`	left to right
9	Assignment	`=` `+= -= *= /=`	Normal assignment Compound assignment: x **+=** 2 means x = x + 2	***right to left***

The system understands **'+'** as string concatenation operator if at least one of its operands is a String. The other operand can be of any type.

Example	Result	Comment
"cat" + "fish"	"catfish"	Concatenate two strings
"" + 1234.56	"1234.56"	A simple way to convert a numeric value to a String
"12" + 34.56	"1234.56"	Concatenate a string and a value of another type

[17] JavaScript "short-circuits" the evaluation of the logical operators. If the left-hand side condition of the || operator is true, then the right-hand side condition of the || operator is not evaluated. If the left-hand side condition of the && operator is false, then the right-hand side condition of the && operator is not evaluated.

1 + 2 + " months" 1 + (2 + " months")	"3 months" "12 months"	The evaluation order is from left to right Expressions in parentheses will be evaluated first
"1 == 2 is " + (1 == 2)	"1 == 2 is false"	1 == 2 evaluates to **`false`**

Type conversions

In Java, ***implicit type conversions*** can occur in the following ways: *promotion, assignment conversion,* and *autoboxing/autounboxing*[18]. In addition, we can do ***explicit type conversions*** (*type casting*) with the cast operator. A type conversion always results to a new separate value.

Although Java is a ***strongly typed*** programming language, some binary operations on values of different type are allowed. In such situations Java *implicitly converts* the operand of a *narrower type to a wider type* to be able to perform the binary operation.

Example	Result	Type of the result	Comment
5 + 4.25	9.25	`double`	first, int is ***promoted*** to double, then addition
1 / 2.0	0.5	`double`	first, int is ***promoted*** to double, then decimal division
1.0 / 2.0	0.5	`double`	decimal division (*no type conversion)*
1 / 2	**0**	**`int`**	integer division (*no type conversion)*

Java allows only *widening conversions (promotion*[19]*)* through assignment.

```
double price = 10;            // Allowed: The int is promoted to double before assignment
int x = 1.25;                 // NOT allowed: double is a wider type than int
int charCode = 'A';           // Allowed: The char is promoted to int before assignment
System.out.print(charCode);   // Prints the character code of 'A', that is 65
```

We can use the *type cast operator* to do an explicit type conversion.

```
double doubleValue = 2.9999;
int intValue = (int)doubleValue;      // Explicit type casting from double to int.
                                      //   => The decimal part is cut off with NO rounding!
System.out.print((double)intValue);   // Explicit type casting from int to double, prints 2.0
```

An implicit type conversion can also occur during a method call.

```
double x = Math.sqrt(2.4);      // The sqrt method expects a double as parameter
double y = Math.sqrt(9);        // Allowed: int is promoted to double
```

Computer arithmetic differs from school arithmetic

The result of an *integer division* is always an integer! There is no rounding in integer division.

```
System.out.println(10 / 9);      // Prints 1
System.out.println(2 / 4) ;      // Prints 0
```

Data types have limited ranges of values. Primitive integer types *overflow silently*!

```
System.out.println(3 * 1431655770);          // Prints 14
System.out.println(Integer.MAX_VALE + 1);    // Prints -2147483648
```

We can also perform some arithmetic operations on characters.

```
for (char letter = 'A'; letter <= 'Z'; letter++) {
    System.out.println(letter);
}
```

---

[18] *Autoboxing*: `Integer integerObject = 123;` *Autounboxing*: `int x = integerObject;`

[19] The ***promotion*** hierarchy is the following: byte → char → int → long → float → double
byte → short → int → long → float → double
NB! When converting from int or long to float, or from long to double, we might lose some least significant digits.

Floating-point arithmetic may give inaccurate results / Using BigDecimal for precise decimal values

Digital computers *cannot represent all decimal numbers exactly*. Java uses a subset of the *IEEE 754 binary floating point standard* to represent floating point numbers. Floating-point decimal types (e.g. **double**) and floating-point arithmetic are ***NOT 100 % accurate*** since some decimal numbers require an infinite number of digits to be represented (e.g. 1.0 / 3.0) and some decimal numbers are not exactly representable for other reasons (e.g. 0.1)[20]. The use of floating-point arithmetic is a trade-off between range/speed and precision/accuracy.

Some floating-point computations can lead to ***inconsistent results***[21] and cause ***potential faults in the program logic***.

```
System.out.println(0.15 + 0.15);          // Prints  0.3
System.out.println(0.1 + 0.2);            // Prints  0.30000000000000004

for (double x = 0.1; x < 0.8; x = x + 0.1) {  // FAULT: This loop should print only
    System.out.println(x);                    // 7 values, but it prints 8 values!
}                                             // The 8th value is 0.7999999999999999
```

Here are some examples of different ways to avoid the above-mentioned inconsistency.

1. If the programming language provides a *special decimal data type for accurate decimal numbers* (for example, C# contains the **decimal** data type), then we can use it instead of the built-in floating-point data types.
2. If the programming language (e.g. JavaScript) does not provide any support for accurate decimal numbers, then
   a. We can use an integer data type and do monetary calculations in cents. That is, we save 12.45 as 1245. This way, calculations run fast, but it takes more work to design, write and test the program code.
   b. Alternatively, we can use some third party library that provides the required functionality. Usually, it is a little bit tricky to work with this kind of utilities.
3. If the programming language's own API includes a class for manipulating high-precision decimal values, then we can use it instead of the built-in floating-point data types.

The ***java.math*** package contains the ***BigDecimal*** class for manipulating arbitrary-size and arbitrary-precision decimal values. The `BigDecimal`[22] class provides high-precision arithmetic needed in monetary calculations etc. To perform accurate calculations, we can create objects from the BigDecimal class as below. Unfortunately, it is a little bit tricky to work with BigDecimals. The example below shows only some very basic operations.

BigDecimal can deal with theoretically infinitely large and infinitely precise decimal values

```
BigDecimal x = new BigDecimal("0.1");          // Initialize x with 0.1
BigDecimal y = new BigDecimal("0.2");          // Initialize y with 0.2
BigDecimal sum = x.add(y);                     // Compute x + y

System.out.println(sum);                       // Prints  0.3

System.out.println(x.subtract(y));             // Prints  -0.1
System.out.println(x.multiply(y));             // Prints  0.02

if (sum.compareTo(new BigDecimal("0.3")) == 0) {
    System.out.println("The sum is correct!");    // Prints : The sum is correct!
}

// To input an accurate decimal value: First, input a String. Then, create a BigDecimal from the String.
String salaryAsText = input.nextLine().replace(',', '.');
BigDecimal salary = new BigDecimal(salaryAsText);
```

[20] See chapter 18 for more details.
[21] This cannot be accepted in monetary calculations etc.
[22] See https://docs.oracle.com/javase/8/docs/api/java/math/**BigDecimal**.html

13 Conditional statements in Java

Conditional statements (the ***if statement*** and the ***switch statement***) execute or skip other statements on the value of a specified condition (aka *logical expression*). A ***logical expression*** (aka *Boolean expression*) is an expression that results in **`true`** or **`false`**. The **if** statement has the following two formats:

```
if (logicalExpression)
    statement
```

```
if (logicalExpression)
    statement_1
else
    statement_2
```

The if statement syntax requires a single statement after the parenthesized logical expression. Here we can use a statement block. A ***statement block*** allows you to use multiple statements where syntax expects a single statement. A statement block is a sequence of statements enclosed within curly braces.

Example 1. Simple if statement

```
if (grade == 0) {
    outputText = "Retake";
}
```

Example 2. Simple if-else

```
if (age < 18) {
    outputText = "Minor";
} else {
    outputText = "Adult";
}
```

Example 3. Multiple choice (please notice the indentation)

```
if (age < 0) {
    outputText = "The value is too low";
} else if (age < 18) {
    outputText = "Minor";
} else if (age < 65) {
    outputText = "Adult";
} else if (age < 123) {
    outputText = "Senior";
} else {
    outputText = "The value is too high";
}
```

Example 4. Multiple choice with the switch statement.

```
switch (examMark) {
case 3:
    outputText = "Excellent";
    break;
case 2:
    outputText = "Good";
    break;
case 1:
    outputText = "Fair";
    break;
case 0:
    outputText = "Fail";
    break;
default:
    outputText = "Invalid exam mark";
    break;
}
```

Conditional expression

Using the ***conditional operator*** (**`?:`**) we can write a simple conditional in a single line. The operator uses three operands: a condition and two expressions. If the condition is true, then the result is the value of the first expression; otherwise the result is the value of the second expression.

```
public static int max(int a, int b) {
    return (a > b) ? a : b;
}
```

14 Writing loops in Java

If the repetition control is based on a *counter*, we normally write **for loop** instead of a while loop[23].

```
for (int i = 1; i <= 10; i++) {
    System.out.println(i);
}
```

If the repetition control is based on *general condition*[24] rather than a counter, we normally write a **while loop**[25]. **NB!** Use the following structure to avoid potential problems in the program logic:

Pseudocode	Java example
Get the first item	`System.out.print("Enter letters (empty ends): ");` `letters = input.nextLine();`
while (*an item is available*) {	`while (letters.length() > 0) {`
Process the item	`text += letters;`
Get the next item	`System.out.print("Enter letters (empty ends): ");` `letters = input.nextLine();`
}	}

The above is the standard way to write a *clean while loop*. The disadvantage is that you have to write twice the statement(s) that get the values. But this is the generic solution that works in all possible situations. You should not write any adventurous code that complicates the logic or is based on side-effects[26].

Sentinel-controlled loops / Inputting several values per each repetition

Sometimes a special value called a *sentinel value* is used to indicate end of data entry. In the example below, the program inputs data (*multiplier* and *multiplicand*) from the user until the user enters a sentinel value (*zero*) as the multiplier to indicate that no more values will be entered.

```
int multiplier = Keyboard.readInt("Enter multiplier (zero ends): ");
while (multiplier != 0) {
    int multiplicant = Keyboard.readInt("Enter multiplicand: ");
    System.out.println(multiplier + " * " + multiplicant + " = " +
         (multiplier * multiplicant));

    multiplier = Keyboard.readInt("Enter multiplier (zero ends): ");
}
```

Nested loops

If the body of a loop contains another loop, we have a *nested loop*[27]. For each iteration of the outer loop, the inner loop executes completely. To keep the code clean, we should avoid too many levels of nesting.

```
for (int x = 1; x <= 10; x++) {
    for (int y = 1; y <= 10; y++) {
        System.out.println(x + " * " + y + " = " + (x * y));
    }

    System.out.println();
}
```

Iterating a collection with the enhanced for loop statement

The ***enhanced for statement*** provides a simple way to iterate a collection in first-to-last order without an index. It works with arrays, *ArrayLists*, and other collections that implement the *Iterable* interface.

```
int[] gradeArray = {5, 4, 5, 4};
for (int grade : gradeArray) {
     System.out.println(grade);
}
```

Java assigns successive values from the collection to the variable on the left of the colon. For each value, the body of the loop is repeated.

---

[23] Technically speaking, any loop can be written using a `while` statement or a `for` statement.
[24] In a case of a generic condition, the number of repetitions cannot be known until the loop begins executing.
[25] Most often, we use the `while` statement instead of a `do-while` statement to keep the code clean.
[26] Please notice that in normal situations it is ***not*** good programming practice to use the `break` statement anywhere else but within a `switch` statement. An equivalent loop can always be written without a `break` statement.
[27] There is no practical limit for levels of nesting loops and other control statements.

15 Methods *in a stand-alone Java program class*

A definition of a simple **method** *of a stand-alone Java program class* requires the following items:

1. Method modifiers **public static** for the **main** method; **private static** for other methods
2. Return type **void** if nothing is returned; otherwise, the data type of the return value
3. Method name
4. Comma-separated *parameter list* in parentheses; each parameter has *data type* and *name*
5. Method body enclosed in braces.

In the first example, the **displayGreeting** method is as simple as it can be. The method does not have any input parameters (*its parameter list is empty*) and it does not return any value (*its return type is* **void**).

FirstMethodExample.java

```
public class FirstMethodExample {

    public static void main(String[] args) {
        displayGreeting();
    }

    private static void displayGreeting() {
        System.out.println("Hello!");
    }
}
```

In the second example, the **showMessage** method accepts two input parameters (*one* **String** *and one* **int**). The method does not return any value (*its return type is* **void**).

SecondMethodExample.java

```
public class SecondMethodExample {

    public static void main(String[] args) {
        showMessage("Hello!", 5);
        showMessage("Goodbye!", 2);
    }

    private static void showMessage(String message, int count) {
        for (int i = 1; i <= count; i++) {
            System.out.println(message);
        }
    }
}
```

In the third example, the **celsiusToFahrenheit** method accepts one input parameter of type **double** and it returns a value of type **double** (*its return type is* **double**).

ThirdMethodExample.java

```
public class ThirdMethodExample {

    public static void main(String[] args) {
        double celsius = 20.0;

        double fahrenheit = celsiusToFahrenheit(celsius);
        System.out.println(celsius + "°C is " + fahrenheit + "°F");
    }

    private static double celsiusToFahrenheit(double celsiusDegrees) {
        return (celsiusDegrees * 9) / 5 + 32;
    }
}
```

Passing parameters to methods

In Java, parameters are always passed to methods ***by value***. When a parameter is passed by value, a ***copy of the parameter's value*** is passed to the called method. A method call can pass the following types of values to a method – ***copies of primitive values*** (int, double, char etc.) and ***copies of references to objects***.

```
int age = 25;
System.out.println(age);
```

In the above example, when the `println` method is called, the system makes a copy of the current value (25) of the age variable and passes the copy to the `println` method. The `println` method works exclusively with the copy. That is, it cannot ever access the age variable directly and change its value.

Passing arrays to methods

In Java, arrays are specially treated *objects*. When an array is passed to a method, the ***value*** that is passed to the called method is a ***copy of a reference to the array object***. That is, no copy of the original array is made. With a reference to the original array the called method can access and change values in the array.

ArrayPassingExample.java

```
public class ArrayPassingExample {

    public static void main(String[] args) {
        int[] myArray = new int[50];

        fillArray(myArray, 1);
        printArray(myArray);
    }

    private static void fillArray(int[] array, int value) {
        for (int i = 0; i < array.length; i++) {
            array[i] = value;
        }
    }

    private static void printArray(int[] array) {
        for (int i = 0; i < array.length; i++) {
            System.out.print(array[i] + " ");
        }
    }
}
```

In the above example, the method `fillArray` can fill the given array with the given value.

Method overloading

Methods ***of the same name*** can be defined in the same class as long as they have different parameter lists. When an *overloaded method* is called, the Java compiler selects the appropriate version of the method by matching the method call's parameter list to a method's parameter list.

```
private static void println(String text) {
      System.out.println("String: " + text);
}

private static void println(int intValue) {
      System.out.println("int: " + intValue);
}
```

Below the *overloaded* **`println`** method is called two times with different parameter lists.

```
println("Hello!");  // calls the println method which accepts one String as its input parameter
println(2017);      // calls the println method which accepts one int as its input parameter
```

Return statements

The ***return*** statement ends method execution and specifies a value to be returned to the caller. If the method's result type is *void*, the return statement can be omitted, or it can be used without specifying any value to be returned to interrupt the execution of the method as follows: `return;`

A method may contain several return statements. When the program execution proceeds to one of those return statements, the method execution ends and therefore no other return statement is executed.

Why create and use methods[28]?

The first reason is very clear; some methods are needed to *make Java programs runnable*. Examples of such methods are the ***main*** method in a stand-alone Java program and the ***doGet*** method in a web application. In addition, your program's interface to the outer world and many useful tools are provided in the form of *Java API classes and their methods*. For example, we use the ***println*** method to print to console and the ***nextLine*** method to input text from the console.

Below is a short list of some *generic reasons* why we create and use methods.

- ✓ To *avoid undesired code duplication*
- ✓ To enable *code reuse* across multiple programs
- ✓ Improve *testing*, *maintenance* and *internal quality of the software*
  A method can be separately tested with *automated unit tests*. The objective is to spend less time fixing defects. *Test-driven development* and *code refactoring* aim to improve the software's internal quality.
- ✓ For *top-down programming* and *structuring the code*
  Methods allow you to *organize the logic into several smaller units*. Smaller units are *easier to design, write, test, understand, and maintain*. You can *start the design process from the top* and break a big problem down into smaller sub problems (smaller methods). Then, you can continue breaking these into several smaller methods until you get *down to very simple problems* to be solved. In addition, the work can be divided among a group of programmers who are specialized in different types of programming tasks.
- ✓ For *abstraction* and *modularity*
  For example, from our viewpoint, the `Arrays.sort` method is an *abstraction of a process* that takes an array as an input and sorts the array. In object-oriented programming, public methods provide the ***public interface*** (visible functionality) for an object. Private methods provide the ***internal implementation*** of the functionality. The objective is to hide implementation details from users of the method.

Some rules of thumb

- There is no lower limit on method length. Smallest reasonable methods contain a single statement. Examples of very small methods are getter and setter methods.
- Normally, you should be able to print the code of a method on a single A4 page. If the method length exceeds this, you should consider breaking the method into two or more smaller methods. In addition, to make your code print neatly, the code lines should not be too long.
- If the current number of nested control statements (loops, conditionals etc.) is more than three and the nested structure is not reasonably short and simple, you should consider writing a new private method and moving (at least) the innermost control structure to the new method.

[28] In Java, *subroutines* are always called **methods**. In different programming languages, subroutines are called procedures, functions, methods, or subprograms.

Method library classes

A *method library class* is a class used to store a group of related *static* methods. In the example below, the classes *Keyboard* and *Maths* provide reusable set of general-purpose methods.

ProgramExample.java

```
public class ProgramExample {

    public static void main(String[] args) {

        String name = Keyboard.readString("Enter your name: ");
        int x = Keyboard.readInt("Enter x: ");
        int y = Keyboard.readInt("Enter y: ");

        System.out.println("Hello, " + name);
        System.out.println("max(x, y) is " + Maths.max(x, y));
    }
}
```

To call a static method of a *method library class* you must write the name of the class and a dot before the method name. For example, **Keyboard.***readString*("Enter your name: ")

Keyboard.java

```
import java.util.Scanner;

public class Keyboard {
    private static Scanner input = new Scanner(System.in);

    public static double readDouble(String prompt) {
        System.out.print(prompt);
        return Double.parseDouble(input.nextLine().replace(',', '.'));
    }

    public static int readInt(String prompt) {
        System.out.print(prompt);
        return Integer.parseInt(input.nextLine());
    }

    public static String readString(String prompt) {
        System.out.print(prompt);
        return input.nextLine();
    }
}
```

In this example, the *Maths* class is here just for demonstrating the idea of using several reusable method library classes.

Maths.java

```
public class Maths {
    public static int max(int a, int b) {
        return (a > b) ? a : b;
    }
}
```

When the program class is being compiled in Eclipse, the Java compiler finds the other two classes automatically if all these three classes are in the same namespace in the same Java project. If a class that you want to use is in another namespace, then you must either use the import directive (e.g. `import` **`java.util.`**`Scanner;`) or refer to the class with its namespace, e.g. **`java.util.`**`Scanner input = new` **`java.util.`**`Scanner(System.in);`

16 Array handling

In Java, an array is a ***fixed-size***[29] ***group of variables*** (called elements) in which an individual element is identified by its position in the array, relative to the first element. To refer to a particular element in the array, we specify the name of the reference and the ***index*** (= position number) of the element in the array. An index must be a non-negative integer (and it can be any expression that evaluates to a positive integer)[30].

lotteryNumbers

23	12	7	9	36	2	16
0	1	2	3	4	5	6

The easiest way of creating an array is to use an array initializer. An ***array initializer*** is a comma-separated list of values in braces. The data type for the variable to be used for referencing to the array is a combination of the **data type of the array element** and **[ ]**.

```
String[] cityArray = {"Helsinki", "Stockholm", "Oslo"};     // array of 3 Strings
double[] averageTemperatures = {16.5, 17.3, 20.8};          // array of 3 doubles
int[] lotteryNumbers = {23, 12, 7, 9, 36, 2, 16};           // array of 7 ints

System.out.println(cityArray[1]);                           // prints Stockholm
System.out.println(lotteryNumbers[0]);                      // prints 23
```

Typically, a for loop is used to iterate through array elements.

```
for (int i = 0; i < lotteryNumbers.length; i++) {
    System.out.println (lotteryNumbers[i]);
}
```

Another way to create an array is with the **new** operator[31].

```
String[] cityArray = new String[3];                 // array of 3 Strings
double[] averageTemperatures = new double[3];       // array of 3 doubles
int[] lotteryNumbers = new int[7];                  // array of 7 ints
```

ArrayExample.java

```
import java.util.Arrays;                    // The Arrays class will be used for sorting an array
import java.util.Scanner;

public class ArrayExample {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        String[] countryArray = new String[3];

        for (int i = 0; i < countryArray.length; i++) {
            System.out.print("Enter country name: ");
            countryArray[i] = input.nextLine();
        }

        Arrays.sort(countryArray);

        for (int i = 0; i < countryArray.length; i++) {
            System.out.println(countryArray[i]);
        }
    }
}
```

[29] That is, an array cannot grow or shrink. If you need a dynamic array-like data structure that can grow and shrink, then you can use an *ArrayList*. But you can use the `Arrays.`***`copyOf`*** method to make a new larger copy of an array.
[30] NB! The last element of an array is at the index equal to the value of the array's ***length*** property minus one.
[31] Yes, arrays are specially treated objects in Java. The indexing operator **[ ]** can be used with array objects only.

The **Arrays** class contains a set of useful methods. Here are some examples:

```
int[] originalArray = {3, 1, 5, 4, 2};

int[] newArray = Arrays.copyOf(originalArray, originalArray.length);
Arrays.sort(newArray);

System.out.println(Arrays.toString(originalArray));        // Prints  [3, 1, 5, 4, 2]
System.out.println(Arrays.toString(newArray));             // Prints  [1, 2, 3, 4, 5]
```

17 String handling and regular expressions

Please see the chapter "**Basic data types and variables**" for the introduction to Strings in Java.

In Java, `String` is a class type, not a primitive data type. That is, string occurrences are objects[32]. Therefore, *you should not use any operator to compare strings*. Instead, you use methods as follows:

```
if (email.isEmpty()) {
    System.out.println("Email is missing");
}

if (countryCode.equals("FI")) {
    System.out.println("Finland");
}
```

Strings are *immutable*. That is, the content of a **String** cannot be changed once it is created. When you want to modify an existing string, the method you call for this purpose creates and returns a new String. The **StringBuilder** class can be used for *mutable* sequences of characters[33].

```
String lastName = "  Janes ";
lastName = lastName.trim();                // Remove leading and trailing space
lastName = lastName.toUpperCase();         // Change to uppercase
lastName = lastName.replace('A', 'O');     // Replace letters => The result is "JONES"
```

It is also possible to chain the method calls as below.

```
lastName = lastName.trim().toUpperCase().replace('A', 'O');
```

Below is a list of some methods of the String class. Before each method name you see what type of a value the method returns and in the method parameter list you see if you are required to pass any parameters to the method.

Desired operation	Method of the String class
Determine string length	`int length()`
Check if the string is empty	`boolean isEmpty()`
Compare two strings for equality	`boolean equals(String anotherString)` `boolean equalsIgnoreCase(String anotherString)`
Sequence two strings *this string > another string , returns* **> 0** *this string = another string , returns* **0** *this string < another string, returns* **< 0**	`int compareTo(String anotherString)` `int compareToIgnoreCase(String anotherString)`
Remove leading and trailing whitespace	`String trim()`
Change case	`String toLowerCase()` `String toUpperCase()`

[32] Strings are *specially treated objects* in Java; behind the scenes **Java creates String objects from a string literals** and the '+' operator has special semantics when its operand is a String. A String object can also be created with the new operator, but this is ***not*** a recommended practice: `String lastName = new String("James");`

[33] If there is a *performance issue* in a string-handling algorithm, then a ***string builder*** can offer better performance.

Replace characters	`String replace(String oldStr, String newStr)` `String replace(char oldChar, char newChar)`
Access a single character in a string	`char charAt(int index)`

Search for a substring
if (found) then the index is returned
else ***-1*** *is returned*
	`int indexOf(String substring)`
	`int indexOf(String substring, int startIndex)`
	`int lastIndexOf(String substring)`

Test if the string starts or ends with the specified prefix	`boolean startsWith(String prefix)` `boolean endsWith(String suffix)`
Extract a substring from beginIndex to endIndex - 1	`String substring(int beginIndex, int endIndex)` `String substring(int index)`
Convert the string to a character array	`char[] toCharArray()`
Split the string using a ***regular expression*** The string is split *no more than* (***limit*** - 1) times	`String[] split(String regex)` `String[] split(String regex, int limit)`
Validate with a ***regular expression***	`boolean matches(String regex)`
Replace substrings that match the given ***regular expression***	`String replaceAll(String regex, String newStr)` `String replaceFirst(String regex, String newStr)`

You can test single characters with the methods of the ***Character*** class.

Desired operation	**Method of the Character class**
Check if the character is a digit	`static boolean isDigit(char c)`

Check if the character is a letter	`static boolean isLetter(char c)`
	`static boolean isUpperCase(char c)`
	`static boolean isLowerCase(char c)`

Check if the character is a letter or digit	`static boolean isLetterOrDigit(char c)`
Check if the character is a whitespace character	`static boolean isWhitespace(char c)`

In the example below, each character in a string is tested separately.

CountUpperCaseLetters.java

```
public class CountUpperCaseLetters {
    public static void main(String[] args) {
        String myString = "Version 1.0A";
        int count = 0;

        for (int i = 0; i < myString.length(); i++) {
            if (Character.isUpperCase(myString.charAt(i))) {
                count++;
            }
        }

        System.out.println(count + " uppercase letter(s).");
    }
}
```

You can also **convert a string to an *array of chars*** for more advanced text manipulation.

```
String myString = "dacbef";

char[] charArray = myString.toCharArray();    // Convert the String to a char array
Arrays.sort(charArray);                       // Sort the char array
myString = new String(charArray);             // Convert the char array to a String
                                              // => "abcdef"
```

You can **split a string** around matches of the given limiter. The limiter is not copied to results.

```
String text = "Helsinki,Finland";
String[] textParts = text.split(",");

System.out.println(textParts[0]);          // Prints Helsinki
System.out.println(textParts[1]);          // Prints Finland
```

NB! The ***split*** method consideres the limiter to be a *regular expression*. Some characters have special meaning in regular expressions. For example, the dot means *any character*, the question mark is a *special quantifier* and the vertical bar '|' means *logical or*. If we want to use any of these in a limiter, then we must write a backslash before the actual character in the regular expression. **NB!** In a *Java string*, we must write *two backslashes*.

```
String text = "www.w3c.org";
String[] textParts = text.split("\\.");        // We must write \\. here because
                                               // the dot means 'any character'
for (String s : textParts) {                   // in a regular expression.
    System.out.println(s);
}
```

In the example below, we extract all words from the text using a regular expression.

```
String text = "Hello, how are you?";
String[] wordArray = text.split("\\s{1,}|,\\s{0,}|\\?");

for (String word : wordArray) {
    System.out.println(word);
}
```

Validating strings with regular expressions

A ***regular expression*** (regex)[34] is a *sequence of characters that define a text-matching pattern*[35]. You can use regex to match, locate, and manage text. In Java, the ***matches*** method tells whether or not this ***entire*** **string matches the given regular expression**[36].

```
if (animal.matches("[bcr]at|cow|horse")) {
    System.out.println("The animal is a bat, cat, rat, cow, or horse.");
}
```

The above condition is true only if the string is "bat", "cat", "rat", "cow", or "horse". For example, if the string is "cats", " cat", "cat ", "Cat", "cats" or "horsepower", then the condition is false.

```
if (postCode.matches("[0-9]{5}") == false) {
    System.out.println("A post code must be exactly 5 digits.");
}
```

The above condition is true only if the length of the string is five and it contains digits only.

```
if (nickName.matches("[A-ZÅÄÖ][A-ZÅÄÖa-zåäö]{3,14}") == false) {
    System.out.println("A nick name should start with an uppercase " +
        "letter and contain between 4 and 15 letters.");
}
```

The Finnish SSN has 11 characters: 6 digits, either '-' or 'A', 3 digits, and one control character.

```
if (ssn.matches("[0-9]{6}(-|A)[0-9]{3}[0-9A-FHJ-NPR-Y]") == false) {
    System.out.println("Invalid social security number format.");
}
```

[34] See https://en.wikipedia.org/wiki/Regular_expression. The package java.util.regex contains more regex tools.
[35] Regex patterns are for ***character matching*** *only*. In regex, there are no semantics for validating numeric ranges.
[36] **NB!** In Java, the *matches* method returns true only **if the *entire string* can be matched**. This is different from some other regex implementations, where the method returns true if the regex can be matched *anywhere* in the string.

Here are some basic regex features that we can use in string matching in Java[37].

User-definable character classes (= character sets defined within square brackets)		

`[first]`
`[a-z]`
`[0-9]`
`[1-47]`
`[a-zåäö]`
`[a-zA-Z]`
	Matches a character that is f, i, r, s, or t
	Between a-z
	Any digit
	1, 2, 3, 4, or 7
	Between a-z, or å, ä, or ö
	Between a-z or between A-Z
		Matches any character inside the brackets
		Character range
		Alternatively, we can write **`\d`**
		Regex is for character matching only
		The Scandinavian character set
		Union

`[^a]`
`[^abc]`
`[^0-9]`
`[a-z&&[^io]]`
	Matches a character that is ***not*** a
	Matches a character that is ***not*** a, b, or c
	Matches a character that is ***not*** a digit
	Between a-z, ***except*** i and o
		A caret (^) *after the opening bracket* negates the *whole* character set.
		Alternatively, we can write **`\D`**
		Subtraction

Pre-defined character classes					
`\s`	A whitespace character	`\d`	A digit	`\w`	A word character: [a-zA-Z_0-9]
`\S`	A non-whitespace character	`\D`	A non-digit	`\W`	A non-word character: [^a-zA-Z_0-9]
`\b`	Word boundary	`.*`**`\b`**`Java`**`\b`**`.*` matches "A cup of Java!", but not "A JavaScript code"			
`.`	Any single character except a line break. NB! Inside square brackets `[.]` the dot is a normal literal.				

Specifying modes inside a regular expression		
`(?i)cat`	Matches *cat, Cat, CAT* ...	`(?i)` turns *case-insensitivity* on and `(?-i)` turns it off
Grouping with parentheses		
`Sun(day){0,1}`	Matches *Sun* and *Sunday*	Alternatively, we can write `Sun(day)`**`?`**
Logical OR		
`(cat|dog)s`	Matches *cats* and *dogs*	

Quantifiers (greedy)	**NB!** A quantifier allows the repetition of the **previous character, character set, or group**	
`X{3}` `[0-9]{5}`	Matches exactly 3 occurrences of X Matches exactly 5 occurrences of a digit	Matches only "XXX" E.g. matches "00520", but not "0520"

`X{0,}`
`X{1,}`
`(Java ){3,}`
`.{0,}zz.{0,}`
	Matches zero or more occurrences of X
	Matches one or more occurrences of X
	Matches 3 or more occurrences of "Java "
	To check if "zz" appears in the text
		Alternatively: `X`**`*`**
		Alternatively: `X`**`+`**
		E.g. matches `"Java Java Java "`
		E.g. matches "One pizza, please"

`X{0,1}`
`X{2,6}`
`colou{0,1}r`
	Matches zero or one occurrence of X
	Matches between 2 and 6 occurrences of X
	Matches *color* and *colour*
		Alternatively: `X`**`?`**
		
		Alternatively, we can write `colou`**`?`**`r`

In the Java context, the regex language includes the special *metacharacters* listed below.

`[]`	Character set	**`|`**	Logical OR	**`?`**	Zero or one time	**`^`**	The start of the line
`{}`	Repetition	**`.`**	Any character	**`*`**	Zero or more times	**`$`**	The end of the line
`()`	Grouping	**`\`**	Escape character	**`+`**	One or more times		

NB! To match a character that has a special meaning in regex, we must write the backslash ***escape character*** before the actual character[38]. For example, **`\.`** matches the dot and **`\\`** matches the backslash (**`\`**).

NB! To insert a backslash ***in a Java string*** you must *precede it by another backslash*. E.g. **`"\\."`**

Replacing substrings with a regular expression (replaceAll, replaceFirst)

The *replaceAll* method replaces each substring of this string that matches the given regular expression with the given replacement. The method *replaceFirst* replaces the first substring only. In the example below, we remove all non-digits from a string.

```
String text = " Abc 12 3e ";
text = text.replaceAll("[^0-9]", "");     // => "123"
```

[37] Unfortunately, there are several regex dialects that provide a little bit different sets of features. This presentation focuses on ***regex in Java*** and introduces only *some selected basics of regex* without any performance considerations. Examples of more advanced regex features are "*boundary matches*", "*backreferences*", "*non-capturing parentheses*", "*reluctant and possessive quantifiers*", and "*look-ahead and look-behind*" (see https://www.regular-expressions.info).

[38] Inside square brackets (in a character class) most of the regex metacharacters are treated as actual literals, but the following must be preceded by the escape character: `\`, `[`, and `]`. In addition, an actual hyphen (`-`) should either be preceded by the escape character or placed at the first or last position inside the brackets to avoid making a range.

18 All primitive data types and internal representations

Primitive types in Java[39]

Data type category		Type	Internal representation of the data type	Example
Numeric data	*One byte*	`byte`	**8** bit *integer* Range: ± 2^7 (-128 ... **127**)	`byte` b = 65;
	Short Integer	`short`	**16** bit *integer* Range: ± 2^{15} (-32768 ... **32767**)	`short` x = 65;
	Integer	`int`	**32** bit *integer* ~ 9-10 digits (± *2 billion*) Range: ± 2^{31} (-2147483648 ... **2147483647**)	`int` y = 65;
	Long Integer	`long`	**64** bit *integer* ~ 17-18 digits Range: ± 2^{63} (± **9223370368547758 07**)	`long` z = 650000000L;
	Decimal number	`float`	**32** bit single-precision IEEE 754 *floating point* ~ 6 digits of precision, not accurate	`float` f = 65.0f;
		`double`	**64** bit double-precision IEEE 754 *floating point* ~ 15 digits of precision, not accurate	`double` d = 65.0;
Textual data	*Single character*	`char`	**16** bit Unicode character	`char` c1 = 'A'; `char` c2 = 65;
Truth value	*Boolean*	`boolean`	**1** bit (1 = true, 0 = false) Allowed values: `true` and `false`	`boolean` b = `true`;

Java classes (In addition, Java provides classes for *date and time*, and *wrapper classes* for all primitive types)

Data type category		Java Class	
Textual data	*String*	`String`	Sequence of **0 – n** Unicode characters
Numeric data	*Big decimal*	`BigDecimal`	Arbitrary-size and arbitrary-precision decimal number
	Big integer	`BigInteger`	Arbitrary-size integer

The result of an integer division is an integer!

```
System.out.println(3 / 2);                    // Prints 1
System.out.println(3 / 6) ;                   // Prints 0
```

Primitive integer types overflow silently!

```
int x = 1431655770 * 3;
System.out.println("1431655770 * 3 = " + x);   // Prints 1431655770 * 3 = 14
```

Floating-point types are not 100 % accurate!

```
double x = 2.0 - 1.1;
if (x == 0.9) {
    System.out.println("2.0 - 1.1 = 0.9");
} else {
    System.out.println("2.0 - 1.1 = " + x);   // Prints 2.0 - 1.1 = 0.8999999999999999
}
```

*We can use immutable **BigDecimal**[40] objects when maximum accuracy is required.*

```
BigDecimal a = new BigDecimal("2.0");
BigDecimal b = new BigDecimal("1.1");
BigDecimal result = a.subtract(b);
System.out.println("2.0 - 1.1 = " + result);   // Prints 2.0 - 1.1 = 0.9

a = new BigDecimal("1000000000000.0");          // The maximum value is almost unlimited
a = a.setScale(4, BigDecimal.ROUND_HALF_UP);    // Number of decimal places and rounding

b = new BigDecimal("6.0");

result = a.divide(b, BigDecimal.ROUND_HALF_UP); // Rounding behavior must be specified

System.out.println(result);                     // Prints 166666666666.6667
```

[39] See https://en.wikibooks.org/wiki/Java_Programming/**Primitive_Types**

[40] The *java.math* package contains the *BigDecimal* and *BigInteger* classes for manipulating arbitrary-size and arbitrary-precision floating-point and integer values. The *BigDecimal* class provides high precision (*and slow*) arithmetic needed in monetary calculations etc. See https://docs.oracle.com/javase/8/docs/api/java/math/**BigDecimal**.html

Internal representation of some primitive data types

Java has the following four types of **integers**: *byte*, *short*, *int* and *long*. The leftmost bit in their internal representation is called the ***sign bit***. If it is zero, the number is positive; otherwise, the number is negative. For example, a *byte* uses one bit for the sign and seven bits to store the value. Therefore, it can represent values between $-(2^7)$ and 2^7-1 or between -128 and 127.

Type	Size	Decimal value	Binary representation	
byte	8b	1 25 65 127 -128 -127	0000 000**1** 000**1** 1001 0100 000**1** 0**111 1111** **1**000 0000 **1**000 000**1**	We can use the tool below to manually convert a positive binary number to a decimal number: 2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0 **128 64 32 16 8 4 2 1** 000**1** 1001 => **16** + **8** + **1** = **25**
short	16b	127 128 -128	0000 0000 0**111 1111** 0000 0000 **1**000 0000 **1111 1111** 1000 0000	
int	32b	7	0000 0000 0000 0000 0000 0000 0000 0**111**	
long	64b	8	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 **1**000	

The integer types are stored in *two's complement* format. To represent a ***negative binary number***, we must first invert all bits of the corresponding positive binary number and then add one to the result. For example, the 8-bit representation of 127_{10} is 0**111 1111**. The value of -127_{10} is represented by first inverting all of the bits of 0**111 1111**, yielding **1**000 0000, then adding 1, yielding **1**000 000**1**

The ***char*** type represents a single 16-bit Unicode character. A ***character set*** is a defined list of characters where each character is represented by its ***code value***. When we assign a character to a variable, Java stores the character's code value in the variable.

Character	Code value	Binary representation
/	47	0000 0000 0010 **1111**
0	48	0000 0000 0011 000**0**
1	49	0000 0000 0011 000**1**
...	...	...
A	65	0000 0000 0100 000**1**
B	66	0000 0000 0100 0010
...	...	...
a	97	0000 0000 0110 000**1**
b	98	0000 0000 0110 0010

Code values **48** to **57**: characters '**0**' to '**9**'
Code values **65** to **90**: characters '**A**' to '**Z**'
Code values **97** to **122**: characters '**a**' to '**z**'

```
char ch = 'A';                    // Java stores 65, the
                                  // code value of 'A'
System.out.println(ch);           // Prints  A
System.out.println((int)ch);      // Prints  65
System.out.println((char)66);     // Prints  B
```

The code below prints a set of characters based on their code values.

```
for (int codeValue = 32; codeValue <= 255; codeValue++) {
    System.out.println(codeValue + "  " + (char)codeValue);
}
```

Java has two **floating-point data types**: *double* (64b) and *float* (32b). *double* has more precision than *float*. Values of the *floating-point* types are stored with the ***sign***, the ***mantissa*** and the ***exponent***. Floating-point types and floating-point arithmetic are *not 100 % accurate*[41].

[41] Advanced information: An IEEE754 compliant 32-bit base-2 floating-point value has a ***sign bit***, an ***exponent*** (8b), and a normalised ***mantissa*** (23b). Shown in the *normalized scientific notation*, the floating-point representation of the decimal number 1.0 is 1.00000000000000000000000 × 2^0 and 8.0 is 1.00000000000000000000000 × 2^3.

For example, the decimal number **0.1** *cannot be represented exactly* in the floating-point format. The approximate floating-point representation of the decimal number 0.1 is 1.10011001100110011001101 × 2^{-4}. Note: If the base was 10 instead of 2, then the decimal number 0.1 could be represented exactly as 1.00000000000000000000000 × 10^{-1}.
=> See the interactive animation of floating-point internals: https://www.h-schmidt.net/FloatConverter/IEEE754.html

19 Packages

In Java, a ***package*** is collection of related classes and interfaces providing access protection and namespace management. Related Java classes can be bundled into packages to make the classes easier to find and use, to avoid class name conflicts[42], and to control access to individual classes.

The ***fully qualified name*** of a class includes the package name. For example, the fully qualified names of the *Scanner* and *System* classes are *java.util.Scanner* and *java.lang.System*. For example, we can use the *Scanner* class without importing it if we use its fully qualified name in the code.

```
java.util.Scanner input = new java.util.Scanner(System.in);
```

By default, the Java compiler knows only the package ***java.lang***. This package contains, for example, the classes *System*, *String*, *Integer*, *Double*, *Math* etc. The ***import*** directive is used to tell the compiler about other packages. For example, to allow the use of the *Scanner* class from the ***java.util*** package we write the following:

```
import java.util.Scanner;
```

We can also allow the use of all classes from a package as follows:

```
import java.util.*;
```

A package name is the same as the folder name that contains the ***.java*** files. Usually, you can create packages in the IDE (e.g. in Eclipse) and the IDE takes care of creating the new folder. In the Java source code, the ***package*** keyword specifies the package where the class belongs. It must be the first non-comment text in the source file.

In the example below, the class ***MyClass*** belongs to a package called ***mypackage***.

MyClass.java

```
package mypackage;

public class MyClass {

}
```

If the Java source file does not contain the package keyword, the class is said to be located in the ***default package'*** It is easy to use the default package, because you do not need to explicitly specify and import it. Using the default package is useful when you are learning Java, but you should not use it later when you are writing code professionally.

[42] For example, the standard Java class library includes two different versions of the *Date* class: one in the *java.util* package and the other in the *java.sql* package. Their fully qualified names are *java.util.Date* and *java.sql.Date*. When we need to use the *Date* class, we import either *java.util.Date* or *java.sql.Date*, depending on the context.

List of Java keywords and Java API classes used in the examples

Java keywords used in the examples

```
import      package
class       private     public      static      void
boolean     byte        char        double      int         long    short
final
if          else        for         while       return      try     catch
new
```

Java API classes / methods used in the examples

Class	Method
System	print println
DecimalFormat	format
Scanner	nextLine close
Double	parseDouble
Integer	parseInt
Arrays	copyOf sort toString
String	compareTo compareToIgnoreCase equals equalsIgnoreCase isEmpty length matches replace replaceAll split toCharArray trim
Character	isdigit isLetter isLowerCase isUpperCase isWhitespace
BigDecimal	add, subtract, multiply, divide, setScale
NumberFormatException	getMessage

Java operators used in the examples

Arithmetic operators
```
+   -   *   /   %
++  --
```

Assignment operators
```
=   +=
```

Relational operators
```
==  !=  >  >=  <  <=
```

Logical operators
```
&&  ||  !
```

String concatenation operator
```
+
```

Conditional operator
```
?:
```

Type cast operator
(*type*)

Note: You can find

All keywords (68 in Java 24) in
List of Java keywords - Wikipedia

All operators (45 in Java 24) in
Java syntax - Wikipedia

Java API classes (ca. 5000 in Java 24)
docs.oracle.com/javase/8/docs/api
docs.oracle.com/en/java/javase/24/docs/api

NB! *Please make sure that you understand the meaning of each item mentioned above.*